ОБЪЕДИНЕНИЕ ТАБЛИЦ

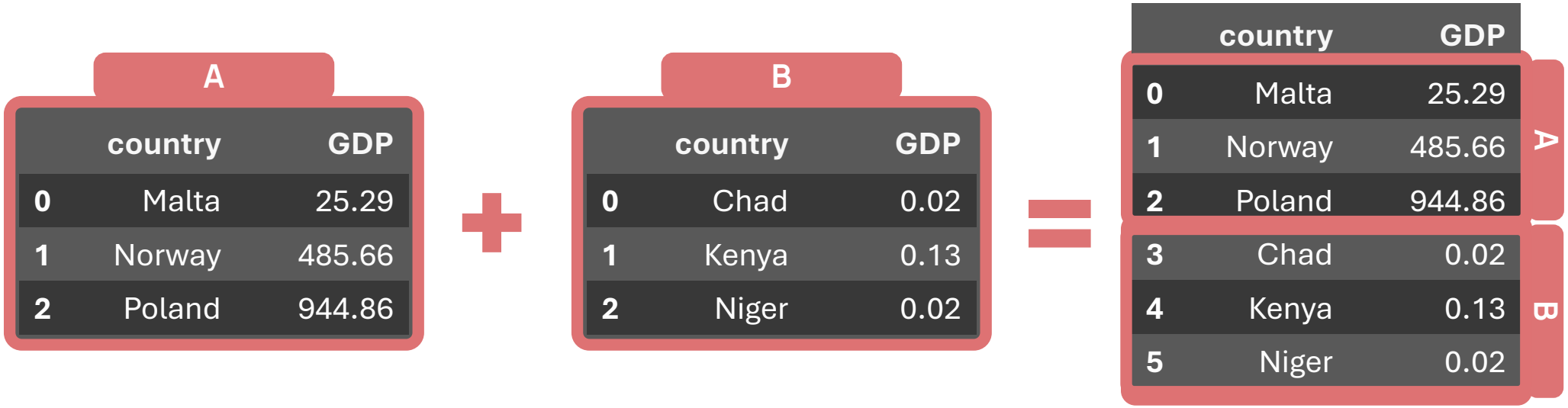
ВЕРТИКАЛЬНОЕ ОБЪЕДИНЕНИЕ (1)

```
pd.concat([датафрейм 1, датафрейм 2, ...])
```

КЕЙС

В таблице **A** хранятся данные по экономическим показателям стран Европы, в таблице **B** хранятся данные по экономическим показателям стран Африки. **Нужно создать общую таблицу, где будут показатели Европы и Африки.**

```
pd.concat([A, B])
```



ВЕРТИКАЛЬНОЕ ОБЪЕДИНЕНИЕ (2)

КЕЙС

В таблице **A** хранятся данные по экономическим показателям стран Европы, в таблице **B** хранятся данные по экономическим показателям стран Африки. **Нужно создать общую таблицу, где будут показатели Европы и Африки.**

```
pd.concat([A, B])
```

A

	country	GDP
0	Malta	25.29
1	Norway	485.66
2	Poland	944.86

+

B

	name	GDP
0	Chad	0.02
1	Kenya	0.13
2	Niger	0.02

=

	country	GDP	name	
0	Malta	25.29	NaN	A
1	Norway	485.66	NaN	A
2	Poland	944.86	NaN	A
3	NaN	0.02	Chad	B
4	NaN	0.13	Kenya	B
5	NaN	0.02	Niger	B

ПЕРЕИМЕНОВАТЬ СТОЛБЦЫ

```
датафрейм = датафрейм.rename(columns={'старое название 1': 'новое название 2', ...})
```

```
B = B.rename(columns={'name': 'country'})
```

B

	name	GDP
0	Chad	0.02
1	Kenya	0.13
2	Niger	0.02

→

B

	country	GDP
0	Chad	0.02
1	Kenya	0.13
2	Niger	0.02

```
pd.concat([A, B])
```

A

	country	GDP
0	Malta	25.29
1	Norway	485.66
2	Poland	944.86

+

B

	country	GDP
0	Chad	0.02
1	Kenya	0.13
2	Niger	0.02

=

	country	GDP	
0	Malta	25.29	A
1	Norway	485.66	A
2	Poland	944.86	A
3	Chad	0.02	B
4	Kenya	0.13	B
5	Niger	0.02	B

ГОРИЗОНТАЛЬНОЕ ОБЪЕДИНЕНИЕ

```
pd.merge(левый датафрейм, правый датафрейм, якорь, how=…)
```



ЯКОРЬ

Якорь - это общая метка по которой можно объединить данные

- Если в левом датафрейме якорь, это **индекс**: `left_index=True`
- Если в правом датафрейме якорь, это **индекс**: `right_index=True`
- Если в левом датафрейме якорь, это **признак**: `left_on='признак'`
- Если в правом датафрейме якорь, это **признак**: `right_on='признак'`

ПАРАМЕТР `how`

Задает то, КАК объединить таблицы

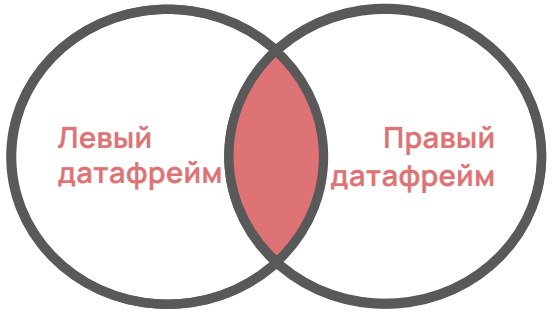
- Если его не указывать или `how='inner'`, останутся только те наблюдения, которые есть в обоих таблицах
- Если нужно сохранить наблюдения только в левой таблице: `how='left'`
- Если нужно сохранить наблюдения только в правой таблице: `how='right'`
- Если нужно сохранить наблюдения из обоих таблиц: `how='outer'`

ПАРАМЕТР how

```
pd.merge(левый датафрейм, правый датафрейм, якорь, how=…)
```

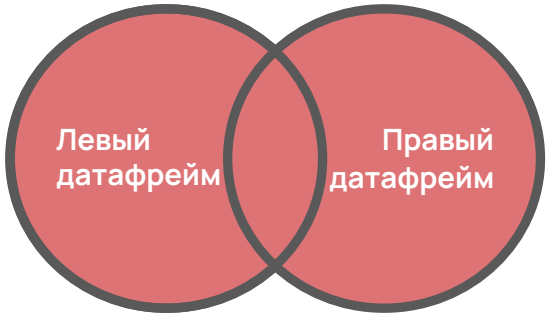


`pd.merge(…, how='inner')`



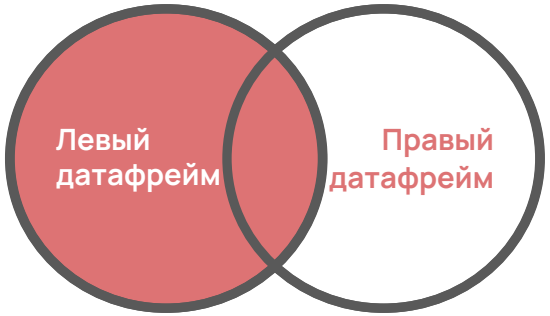
Останутся только те наблюдения, которые есть в обоих таблицах одновременно

`pd.merge(…, how='outer')`



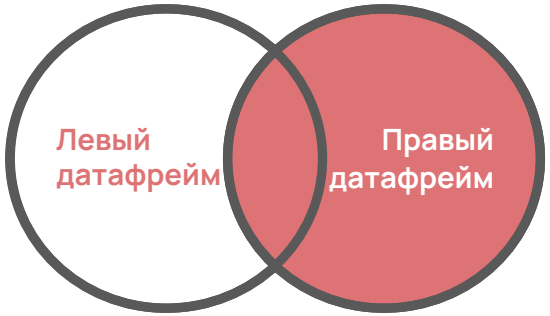
Останутся наблюдения из обоих таблиц

`pd.merge(…, how='left')`



Останутся наблюдения из левой таблицы

`pd.merge(…, how='right')`



Останутся наблюдения из правой таблицы

КЕЙС. ПАРАМЕТР `how`

КЕЙС

В таблице **A** хранятся данные по уровню ВВП стран Европы. В таблице **B** хранится численность населения стран Европы. Аналитик данных написал следующий код, чтобы объединить данные по странам из двух таблиц.
Укажите, какой параметр `how` , был использован в каждом случае:

```
pd.merge(A, B, …,how=?)
```

A

	country	GDP
0	Malta	25.29
1	Norway	485.66
2	Poland	944.86

B

	country	pop
0	Norway	5.62
1	Estonia	1.34
2	Greece	9.94

РЕЗУЛЬТАТ 1

	country	GDP	pop
0	Malta	25.29	NaN
1	Norway	485.66	5.62
2	Poland	944.86	NaN

РЕЗУЛЬТАТ 2

	country	pop	GDP
0	Norway	5.62	485.66

РЕЗУЛЬТАТ 3

	country	pop	GDP
0	Norway	5.62	485.66
1	Estonia	1.34	NaN
2	Greece	9.94	NaN

РЕЗУЛЬТАТ 4

	country	GDP	pop
0	Malta	25.29	NaN
1	Norway	485.66	5.62
2	Poland	944.86	NaN
3	Estonia	NaN	1.34
4	Greece	NaN	9.94

ЯКОРЬ

```
pd.merge(левый датафрейм, правый датафрейм, якорь, how=…)
```

Якорь - это общая метка по которой можно объединить данные:

```
left_index=True, right_index=True
```




```
left_on='признак', right_on='признак'
```




```
left_index=True, right_on='признак'
```




```
left_on='признак', right_index=True
```



КЕЙС. ЯКОРЬ

КЕЙС

В таблице **A** хранятся данные по уровню ВВП стран Европы. В таблице **B** хранится численность населения стран Европы. Аналитик данных написал следующий код, чтобы объединить данные по странам из двух таблиц.
Укажите, что будет якорем в каждом случае и как это оформить в коде:

```
pd.merge(A, B, якорь?)
```

ВАРИАНТ 1

A

	country	GDP
0	Malta	25.29
1	Norway	485.66
2	Poland	944.86

B

	pop
Norway	5.62
Estonia	1.34
Greece	9.94

ВАРИАНТ 2

A

	GDP
Malta	25.29
Norway	485.66
Poland	944.86

B

	pop
Norway	5.62
Estonia	1.34
Greece	9.94

ВАРИАНТ 3

A

	country	GDP
0	Malta	25.29
1	Norway	485.66
2	Poland	944.86

B

	name	pop
0	Norway	5.62
1	Estonia	1.34
2	Greece	9.94

ВАРИАНТ 4

A

	GDP
Malta	25.29
Norway	485.66
Poland	944.86

B

	name	pop
0	Norway	5.62
1	Estonia	1.34
2	Greece	9.94

B3_A

Информация о предложениях аренды жилья на сайте Airbnb в Нью-Йорке. Датасет содержит следующие признаки:

train_B
id - номер объявления
last booking - дата последнего бронирования
price - стоимость одной ночи ($)
host name - имена хозяев (если хозяев несколько, они перечислены через запятую с пробелом)

Вар 1: Для каждого объявление в таблице train_B найдите район (**area**; train1_B3), в котором предлагается аренда, и размер сервисного сбора (**service fee**; train1_B3).

train1_B3
id rent - номер объявления
area - район, в котором расположено жилье
service fee - размер сервисного сбора ($)

Вар 3: Для объявлений, которые есть одновременно в train_B и train3_B3 найдите стоимость проживания (**price**; train_B), район, где расположены апартаменты (**area**; train3_B3) и доступность в течение года (**availability**; train3_B3).

train3_B3
id rent - номер объявления
area - район, в котором расположено жилье
availability - доступность жилья в течение года (в днях)

Вар 4: Укажите, сколько заработает (**price**) хозяин (**host name**), владеющий наибольшим количеством апартаментов, если вся его недвижимость будет сдана одновременно сроком на 10 дней, с учетом объявлений в таблицах train_B и train4_B3.

train4_B3
id - номер объявления
host name - имена хозяев (если хозяев несколько, они перечислены через запятую с пробелом)
price per night - стоимость одной ночи ($)

СВОДНЫЕ ТАБЛИЦЫ

АГРЕГИРОВАНИЕ ДАННЫХ

Агрегирование данных — это процесс объединения множества отдельных единиц данных в одну обобщённую форму для облегчения анализа и понимания общей картины.

```
датафрейм['признак'].метод()
```

- **Минимум →** `.min()` **→** `int64`
- **Максимум →** `.max()` **→** `int64`
- **Сумма→** `.sum()` **→** `int64`
- **Среднее арифметическое →** `.mean()` **→** `int64`
- **Количество не пропущенных значений (не NaN)→** `.count()` **→** `int64`
- **Количество уникальных значений →** `.nunique()` **→** `int64`
- **Самое(ые) частое(ые) значение(ия) →** `.mode()` **→** `Series`

	признак
0	A
1	B
2	NaN
3	A
4	C

```
df['признак'].shape[0]
```

```
5
```

```
df['признак'].count()
```

```
4
```

```
df['признак'].nunique()
```

```
3
```

```
df['признак'].mode()
```

	признак
0	A

```
df['признак'].mode()[0]
```

```
'A'
```

КЕЙС. АГРЕГИРОВАНИЕ ДАННЫХ

КЕЙС

В таблице **A** хранятся характеристики стран Европы. Как ответить на вопросы ниже с помощью агрегирования?

country – название страны
GDP – ВВП
pop – численность населения
gov – система правления

A

	country	GDP	pop	gov
0	Malta	25.29	0.55	par
1	Norway	485.66	5.62	par
2	Poland	944.86	37.98	spr
3	Estonia	NaN	1.34	par
4	Greece	NaN	9.94	par
...	...	...	...	...

1. Какой средний уровень ВВП (**GDP**) стран Европы?
2. Какой наименьший и наибольший ВВП (**GDP**) в Европе?
3. ВВП (**GDP**) скольких стран Европы известно?
4. Какая численность населения (**pop**) Европы?
5. Какой размах* численности населения (**pop**) Европы?
6. Сколько систем правления стран (**gov**) представлено в данных?
7. Какая система правления (**gov**) наиболее популярная в Европе?
8. Какая средняя численность населения (**pop**) в странах с полупрезидентской системой правления (*spr*; **gov**)?

***Размах** – разница между наибольшим и наименьшим значением

СВОДНЫЕ ТАБЛИЦЫ

Сводная таблица - это таблица агрегирующая данные по категориям определенного признака или признаков.

*Пример: По таблице **df** для каждой категории **признака 1** рассчитайте среднее значение **признака 2**.*

Неумный способ 👎:

```
df[df['признак 1'] == 'A']['признак 2'].mean()
df[df['признак 1'] == 'B']['признак 2'].mean()
…
```

Умный способ 👍:

df

	признак 1	признак 2
0	A	10
1	B	5
2	A	20
3	C	10
4	D	20
5	B	15
6	A	30

`.groupby()`



`.pivot_table()`

Сводная таблица

признак 1	признак 2
A	20
B	10
C	10
D	20

ПРОСТЕЙШАЯ ГРУППИРОВКА

```
датафрейм.groupby('признак группировки')['признак агрегирования'].метод()
```

```
датафрейм.pivot_table(index='признак группировки',
                      values='признак агрегирования',
                      aggfunc='метод агрегирования')
```

КЕЙС

На основе таблицы **A,** для каждой системы правления (**gov**) рассчитайте средний уровень ВВП (**GDP**).

A

	country	GDP	pop	gov
0	Malta	25.29	0.55	par
1	Norway	485.66	5.62	par
2	Poland	944.86	37.98	spar
...	...	...	...	...

```
A.groupby('gov')['GDP'].mean()
```

ИЛИ

```
A.pivot_table(index='gov',
              values='GDP',
              aggfunc='mean')
```

Сводная таблица

	GDP
gov	
par	12.8
spar	14.2
pres	7.8

НЕСКОЛЬКО ПРИЗНАКОВ АГРЕГИРОВАНИЯ

```
датафрейм.groupby('признак группировки')[['признак аг 1', 'признак аг 2', …]].метод()
```

```
датафрейм.pivot_table(index='признак группировки',
                      values=['признак аг 1', 'признак аг 2', …],
                      aggfunc='метод агрегирования')
```

КЕЙС

На основе таблицы **A,** для каждой системы правления (**gov**) рассчитайте средний уровень ВВП (**GDP**) и среднюю численность населения (**pop**).

A

	country	GDP	pop	gov
0	Malta	25.29	0.55	par
1	Norway	485.66	5.62	par
2	Poland	944.86	37.98	spar
…	…	…	…	…

```
A.groupby('gov')[['GDP', 'pop']].mean()
```

ИЛИ

```
A.pivot_table(index='gov',
              values=['GDP', 'pop'],
              aggfunc='mean')
```

Сводная таблица

gov	GDP	pop
par	12.8	50.5
spar	14.2	35.6
pres	7.8	2.5

НЕСКОЛЬКО ПРИЗНАКОВ ГРУППИРОВКИ

```
датафрейм.groupby(['признак гр 1', 'признак гр 2', …])['признак агрегирования'].метод()
```

```
датафрейм.pivot_table(index=['признак гр 1', 'признак гр 2', …],
                      values= 'признак агрегирования',
                      aggfunc='метод агрегирования')
```

КЕЙС

На основе таблицы **A,** внутри каждого региона (**reg**), рассчитайте средний уровень ВВП (**GDP**) для каждой системы правления (**gov**).

A

	country	GDP	reg	gov
0	Malta	25.29	S	par
1	Norway	485.66	N	par
2	Poland	944.86	W	spr
…	…	…	…	…

```
A.groupby(['reg', 'gov'])['GDP'].mean()
```

ИЛИ

```
A.pivot_table(index=['reg', 'gov'],
              values='GDP',
              aggfunc='mean')
```

Сводная таблица

		GDP
reg	**gov**	
S	**par**	5.2
	spr	10.4
	pres	2.2
N	**par**	12.7
	…	…

НЕСКОЛЬКО МЕТОДОВ АГРЕГИРОВАНИЯ (1)

```
датафрейм.groupby('признак группировки')['признак агрегирования'].agg(['метод 1', 'метод 2', …])
```

```
датафрейм.pivot_table(index='признак группировки',
                      values= 'признак агрегирования',
                      aggfunc=['метод 1', 'метод 2', …])
```

КЕЙС

На основе таблицы **A**, для каждой системы правления (**gov**) рассчитайте средний и максимальный уровень ВВП (**GDP**).

A

	country	GDP	reg	gov
0	Malta	25.29	S	par
1	Norway	485.66	N	par
2	Poland	944.86	W	spr
…	…	…	…	…

```
A.groupby('gov')['GDP'].agg(['mean', 'max'])
```

ИЛИ

```
A.pivot_table(index='gov',
              values='GDP',
              aggfunc=['mean', 'max'])
```

Сводная таблица

	mean	max
gov		
par	5.2	523.7
spr	10.4	123.4
pres	2.2	628.8

НЕСКОЛЬКО МЕТОДОВ АГРЕГИРОВАНИЯ (2)

```
датафрейм.groupby('признак группировки').agg({'признак аг 1':'метод 1', 'признак аг 2':'метод 2', …})
```

```
датафрейм.pivot_table(index='признак группировки',
                      aggfunc={'признак аг 1':'метод 1', 'признак аг 2':'метод 2', …})
```

КЕЙС

На основе таблицы **A**, для каждой системы правления (**gov**) рассчитайте средний ВВП (**GDP**) и максимальную численность населения (**pop**).

A

	country	GDP	pop	gov
0	Malta	25.29	0.55	par
1	Norway	485.66	5.62	par
2	Poland	944.86	37.98	spr
…	…	…	…	…

```
A.groupby('gov').agg({'GDP':'mean',
                      'pop': 'max'})
```

или

```
A.pivot_table(index='gov',
              aggfunc={'GDP':'mean',
                       'pop': 'max'})
```

Сводная таблица

gov	GDP	pop
par	5.2	83.58
spr	10.4	66.69
pres	2.2	1.37

ПРИМЕНЕНИЕ КАСТОМНОЙ ФУНКЦИИ

```
датафрейм.groupby('признак группировки')['признак агрегирования'].agg(<функция>)
```

```
датафрейм.pivot_table(index='признак группировки',
                      values= 'признак агрегирования',
                      aggfunc=<функция>)
```

КЕЙС

На основе таблицы **A**, для каждой системы правления (**gov**) рассчитайте размах ВВП (**GDP**).

A

	country	GDP	pop	gov
0	Malta	25.29	0.55	par
1	Norway	485.66	5.62	par
2	Poland	944.86	37.98	spr
...	...	...	...	...

```
def coded(x):
  return x.max()-x.min()
```

```
A.groupby('gov')['GDP'].agg(coded)
```

или

```
A.pivot_table(index='gov',
              values='GDP',
              aggfunc=coded)
```

Сводная таблица

	GDP
gov	
par	0.2
spr	1.4
pres	0

B3_B

Информация о предложениях аренды жилья на сайте Airbnb в Нью-Йорке. Датасет содержит следующие признаки:

data
id - номер объявления
area - район, в котором расположено жилье
price - стоимость одной ночи ($)
km2 - площадь района ($км^2$)

Вар 1: Среди районов (**area**), в которых менее 100 предложений об аренде (**id**), найдите район (**area**), в котором средняя стоимость аренды (**price**) наибольшая.

Вар 3: Найдите размах стоимости аренды (**price**) в каждом районе (**area**). Укажите в скольких районах размах стоимости аренды не равен 0.

Вар 5: Найдите район (**area**) с наибольшей плотностью предложений об аренде (**id/km2**). В ответ укажите значение этой плотности. Ответ округлите до сотых.

ВИЗУАЛИЗАЦИЯ

ВИЗУАЛИЗАЦИЯ КОЛИЧЕСТВЕННЫХ ПРИЗНАКОВ

Распределение количественных признаков:

- **Гистограмма**: `Series.plot(kind='hist')`
- **Ящик с усами**: `Series.plot(kind='box')`

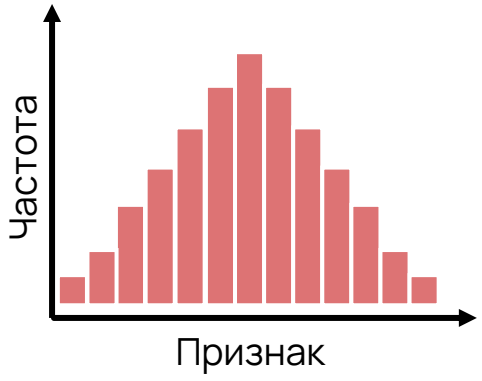
Динамика количественного признака:

- **Линейный график**: `DataFrame.plot(y='признак')`

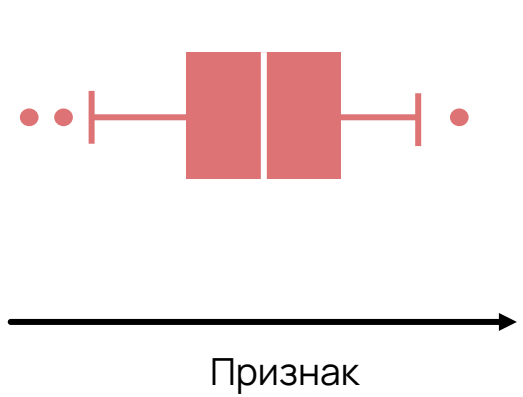
Зависимость двух количественных признаков:

- **Диаграмма рассеяния**: `DataFrame.plot('признак x', 'признак y', kind='scatter')`

ГИСТОГРАММА



ЯЩИК С УСАМИ



ЛИНЕНЫЙ ГРАФИК

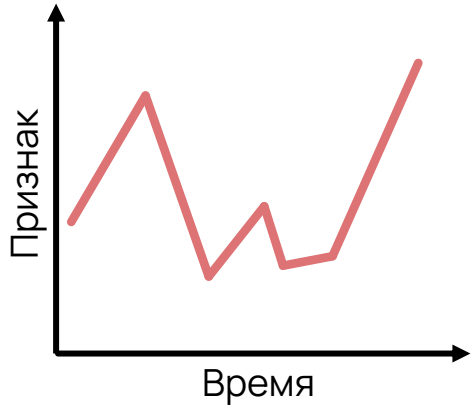
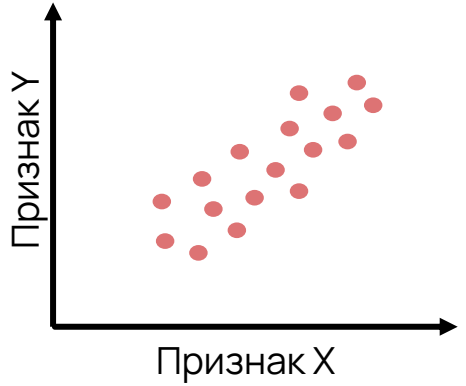


ДИАГРАММА РАССЕЯНИЯ



ГИСТОГРАММА

```
Series.plot(kind='hist', bins=количество интервалов(по дефолту 10))
```

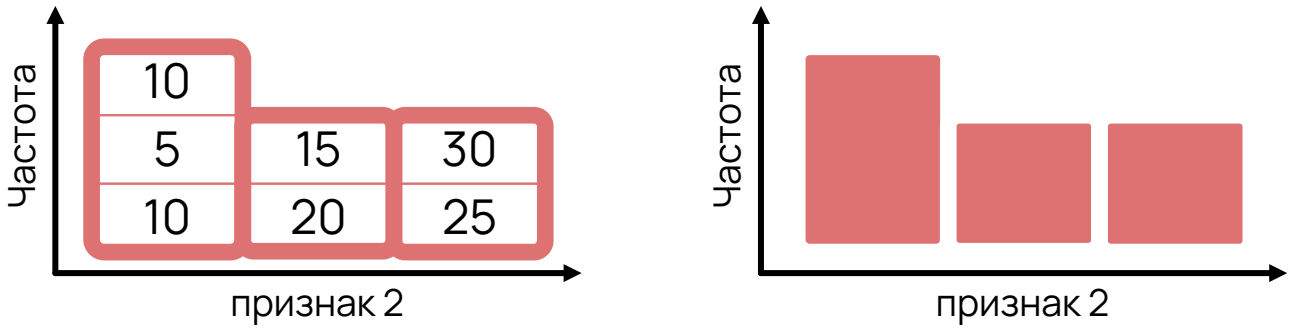
Гистограмма показывает распределение количественного признака разбитое по интервалам.

Пример: Нужно построить гистограмму распределения ***признака 2***.

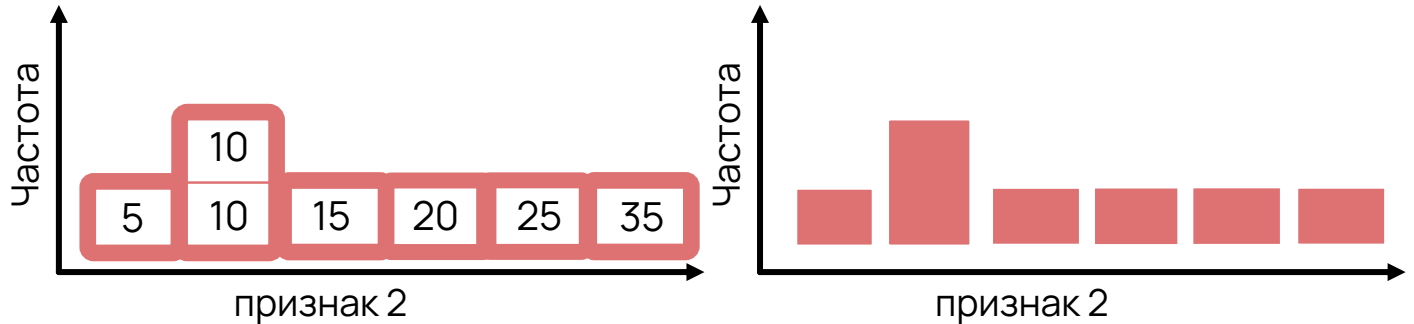
df

	признак 1	признак 2
0	A	10
1	B	5
2	A	20
3	C	10
4	D	25
5	B	15
6	A	30

```
df['признак 2'].plot(kind='hist', bins=3)
```




```
df['признак 2'].plot(kind='hist', bins=6)
```



ЯЩИК С УСАМИ

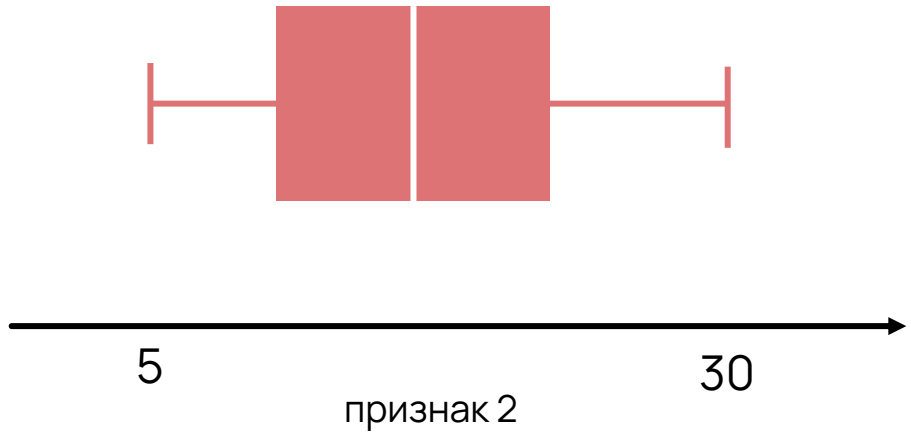
```
Series.plot(kind='box', vert=False)
```

Ящик с усами показывает распределение количественного признака и его основные статистики.
Пример: Нужно построить ящик с усами ***признака 2****.*

df

	признак 1	признак 2
0	A	10
1	B	5
2	A	20
3	C	10
4	D	25
5	B	15
6	A	30

```
df['признак 2'].plot(kind='box', vert=False)
```



ЛИНЕЙНЫЙ ГРАФИК

```
DataFrame.plot(y='признак')
```

Линейный график показывает динамику количественного признака в течение времени.
Пример: Нужно построить линейный график, показывающий динамику ***признака 2*** *за три года*

```
df.plot(y='признак 2')
```

	признак 2
2021	10
2022	5
2023	20

```
df.sort_index().plot(y='признак 2')
```

	признак 2
2023	20
2021	10
2022	5

```
df.sort_values('год').plot(y='признак 2')
```

	год	признак 2
0	2023	20
1	2021	10
2	2022	5

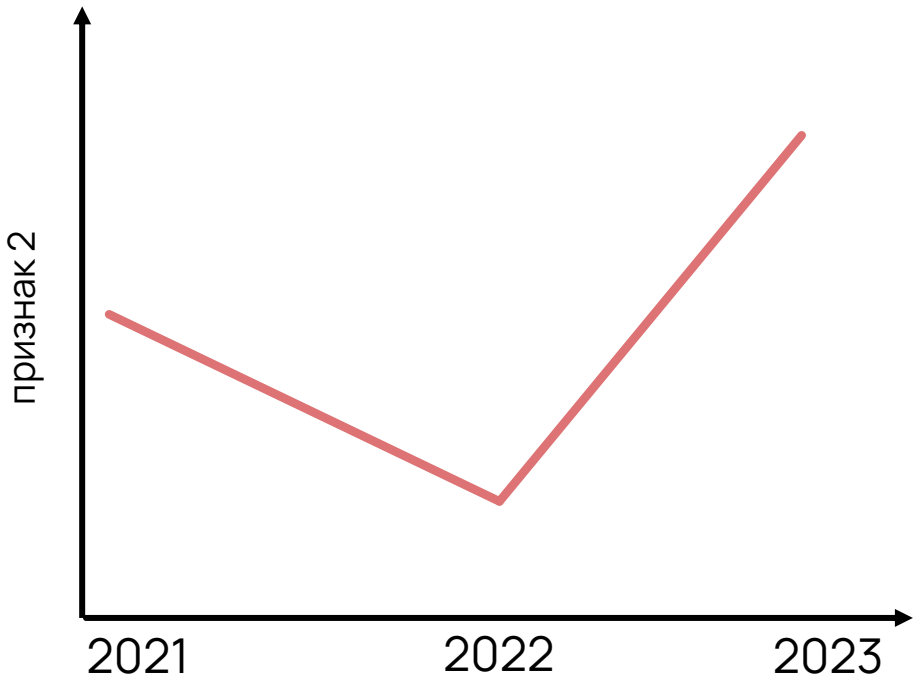


ДИАГРАММА РАССЕЯНИЯ

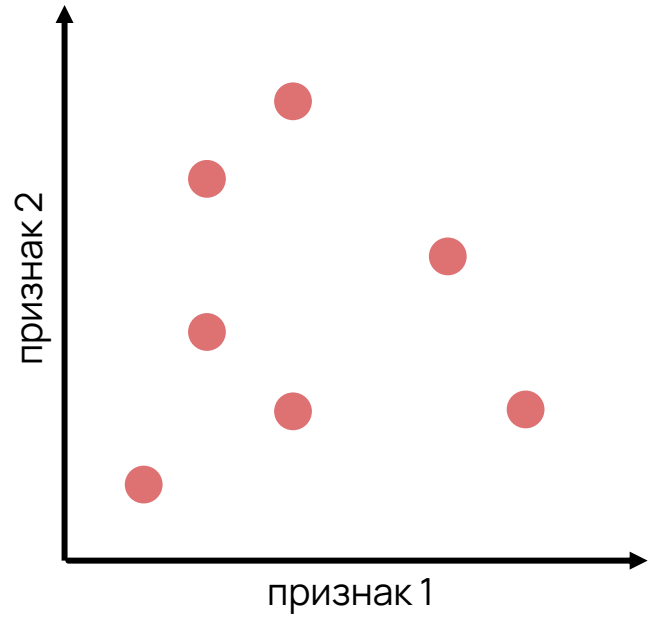
```
DataFrame.plot('признак x', 'признак y', kind='scatter')
```

Диаграмма рассеяния показывает зависимость двух количественных признаков
*Пример: Нужно построить диаграмму рассеяния **признака 1** и **признака 2***

```
df.plot('признак 1', 'признак 2', kind='scatter')
```

df

	признак 1	признак 2
0	15	10
1	5	5
2	25	20
3	30	10
4	10	25
5	10	15
6	15	30



ВИЗУАЛИЗАЦИЯ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

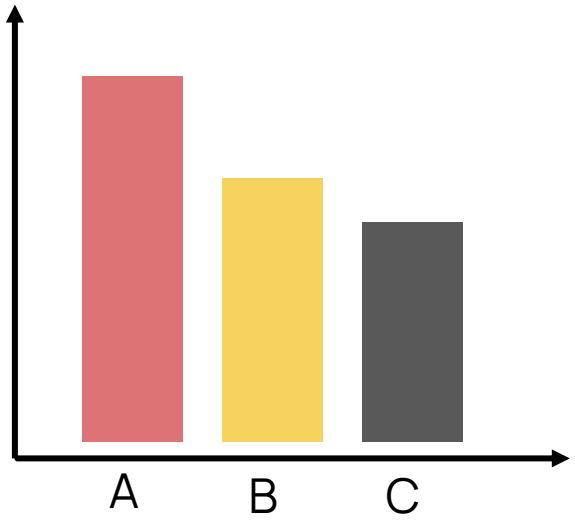
Сравнить категории признака по какому-то показателю:

- **Столбчатая диаграмма**: `DataFrame.plot(y='признак', kind='bar')` или `DataFrame.plot(y='признак', kind='barh')`

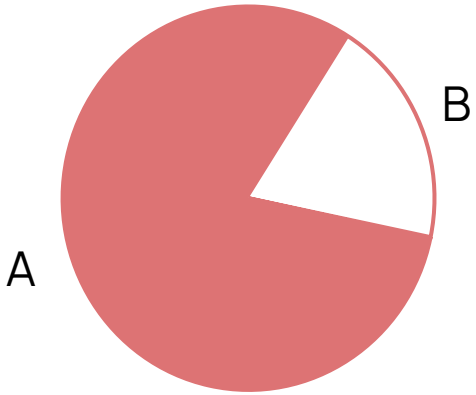
Сравнить доли категорий в составе целого:

- **Круговая диаграмма**: `DataFrame.plot(y='признак', kind='pie')`

СТОЛБЧАТАЯ ДИАГРАММА



КРУГОВАЯ ДИАГРАММА



СТОЛБЧАТАЯ ДИАГРАММА

```
DataFrame.plot(y='количественный признак', kind='bar')
```

```
DataFrame.plot(y='количественный признак', kind='barh')
```

*Пример: Нужно визуализировать среднее значение **признака 2** для каждой категории **признака 1***

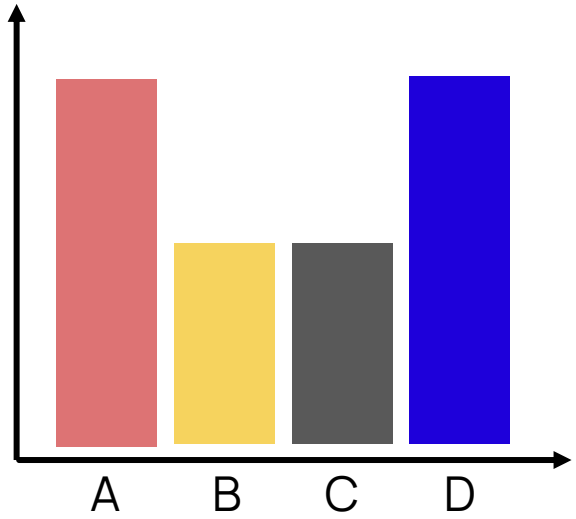
```
gb.plot(y='признак 2', kind='bar')
```

df

	признак 1	признак 2
0	A	10
1	B	5
2	A	20
3	C	10
4	D	20
5	B	15
6	A	30

Сводная таблица (gb)

	признак 2
признак 1	
A	20
B	10
C	10
D	20



КРУГОВАЯ ДИАГРАММА

```
DataFrame.plot(y='количественный признак', kind='pie')
```

*Пример: Нужно визуализировать распределение категорий **признака 1***

df

	признак 1	признак 2
0	A	10
1	B	5
2	A	20
3	C	10
4	D	20
5	B	15
6	A	30

Сводная таблица (gb)

	count
признак 1	
A	3
B	2
C	1
D	1

```
gb.plot(y='count', kind='pie')
```

